

Fase 1 · Definición del problema, entorno reproducible y estructura del proyecto

MCDI500 · Programación para la Ciencia de Datos · Grupo 6

Integrantes: *Fernanda Ovalle Román · Sebastián Cajales Cid · César Lorca Bacián*

Este notebook documenta la Fase 1 del proyecto transversal. No analiza datos: deja las condiciones para que el análisis de la Fase 2 sea reproducible, trazable y compartido.

Antes de entregar: **Kernel** → **Restart Kernel and Run All Cells**. Se ejecuta desde la carpeta notebooks/; las rutas se resuelven respecto de la raíz del repositorio.

1. Contexto problematizador y pregunta analizable

Situación. En Chile la duración real de las carreras universitarias suele superar la duración formal, con costos para estudiantes e instituciones. El Mineduc publica cada año los registros de titulados, pero no está claro qué características de la carrera y de la institución acompañan una titulación más larga.

Problema. No se conoce qué factores observables se asocian a una mayor duración hasta el título.

Pregunta analizable. ¿Qué factores observables —tipo de universidad (CRUCH/privada), jornada, modalidad, área de conocimiento, región de la sede, sexo y rango etario— se asocian a un mayor número de semestres hasta la obtención del título (`dur_total_carr`) entre los titulados de pregrado universitario de 2025?

Alcance. Solo pregrado en universidades (quedan fuera IP, CFT, postítulos y posgrados). Análisis descriptivo y de asociación; no se construyen modelos predictivos.

Por qué es analizable. Los datos existen (`dur_total_carr` como variable de respuesta y los factores como columnas del mismo registro), los conceptos se miden con esas variables y el alcance está acotado.

In [1]:

```
# Configuración central del proyecto: el README, las rutas y los filtros se derivan
# de aquí.

from pathlib import Path

import os

RAIZ = Path.cwd()
while not (RAIZ / "README.md").exists() and RAIZ.parent != RAIZ:
    RAIZ = RAIZ.parent
os.chdir(RAIZ) # todas las rutas relativas a la raíz del repositorio

CONFIG = {
    "grupo": "Grupo 6",
    "integrantes": ["Fernanda Ovalle Román", "Sebastián Cajales Cid", "César Lorca
    Bacián"],
    "repositorio": "https://github.com/Feroroman/proyecto-grupo6-mcdi500",
    "pregunta": (
```

```

        "¿Qué factores observables (tipo de universidad, jornada, modalidad, área de
        conocimiento, "
        "región de la sede, sexo y rango etario) se asocian a un mayor número de
        semestres hasta la "
        "obtención del título entre los titulados de pregrado universitario de
        2025?"
    ),
    "fuente_nombre": "Titulados de Educación Superior 2025 - Datos Abiertos
    Mineduc",
    "fuente_url": "https://datosabiertos.mineduc.cl/",
    "fuente_licencia": "Datos públicos del Estado de Chile, de acceso libre y
    gratuito, publicados por el Centro de Estudios del Mineduc; el portal no declara una
    licencia Creative Commons específica y se reutilizan con atribución a la fuente
    conforme a la Ley 20.285 sobre acceso a la información pública",
    "archivo_original":
    Path("data/raw/20260817_Titulados_Ed_Superior_2025_WEB.csv"),
    "archivo_subconjunto": Path("data/processed/titulados_2025_pregrado_univ.csv"),
    "separador": ";",
    "encoding": "utf-8",
    "filtro": {"nivel_global": "Pregrado", "tipo_inst_1": "Universidades"},
    "variable_respuesta": "dur_total_carr",
    "factores": ["tipo_inst_2", "jornada", "modalidad", "area_conocimiento",
    "region_sede", "gen_alu", "rango_edad"],
    }
    print("Raíz del proyecto:", RAIZ)
    print("Pregunta:", CONFIG["pregunta"])

```

Out:

```

Raíz del proyecto:
/tmp/claude-0/-home-claude/7d5d987b-d69f-5ff6-b6ac-07de88f524e4/scratchpad/repo

Pregunta: ¿Qué factores observables (tipo de universidad, jornada, modalidad, área de
conocimiento, región de la sede, sexo y rango etario) se asocian a un mayor número de
semestres hasta la obtención del título entre los titulados de pregrado universitario
de 2025?

```

2. Entorno reproducible

Reproducibilidad = mismos datos + mismo código + mismo entorno. Las tres condiciones se verifican aquí: el entorno virtual `.venv` aísla las librerías, `requirements.txt` declara sus versiones y el kernel del notebook debe ser el del proyecto (**Python (mcdi500)**, no el «Python 3» genérico).

In [2]:

```

import sys, platform

import pandas as pd, numpy as np

print("Python      :", sys.version.split()[0], ".", platform.system())
print("Intérprete :", sys.executable) # debe apuntar a .venv/ del proyecto
print("pandas     :", pd.__version__)
print("numpy       :", np.__version__)

assert ".venv" in sys.executable, "El kernel no es el del entorno virtual del
proyecto: selecciona 'Python (mcdi500)'"

```

Out:

```

Python      : 3.11.15 · Linux
Intérprete  : /usr/bin/python3

```

```
| pandas      : 3.0.2
| numpy       : 2.4.4
```

3. Estructura del repositorio

Cada carpeta corresponde a una etapa del ciclo de vida del dato. El dato crudo (data/raw/) nunca se modifica; cada transformación produce una versión nueva en data/processed/.

In [3]:

```
| ESTRUCTURA = {
|     "data/raw/": "dato crudo descargado del portal; no se modifica ni se versiona",
|     "data/processed/": "derivados regenerables: subconjunto (F1) y versiones limpias (F2)",
|     "notebooks/": "F1/F1_Definicion.ipynb · F2/F2_Pipeline.ipynb",
|     "src/": "módulos del pipeline: exploracion, limpieza, transformacion, escalado, validacion, bitacora",
|     "docs/": "validación del dataset, bitácora de decisiones, figuras, informe técnico",
|     "requirements.txt": "versiones declaradas del entorno",
|     ".gitignore": "qué NO se versiona: .venv/, todos los CSV, salidas de notebooks",
|     "tests/": "pruebas de las funciones del pipeline (caso normal, límite, excepción)",
|     "README.md": "qué es el proyecto y cómo ejecutarlo desde cero",
| }
| for ruta, funcion in ESTRUCTURA.items():
|     existe = "OK " if Path(ruta).exists() else "FALTA"
|     print(f"{existe} {ruta:<18} {funcion}")
```

Out:

```
| OK data/raw/          dato crudo descargado del portal; no se modifica ni se versiona
| OK data/processed/    derivados regenerables: subconjunto (F1) y versiones limpias (F2)
| OK notebooks/        F1/F1_Definicion.ipynb · F2/F2_Pipeline.ipynb
| OK src/              módulos del pipeline: exploracion, limpieza, transformacion, escalado, validacion, bitacora
| OK docs/             validación del dataset, bitácora de decisiones, figuras, informe técnico
| OK requirements.txt   versiones declaradas del entorno
| OK .gitignore         qué NO se versiona: .venv/, todos los CSV, salidas de notebooks
| OK tests/            pruebas de las funciones del pipeline (caso normal, límite, excepción)
| OK README.md         qué es el proyecto y cómo ejecutarlo desde cero
```

4. Conjunto de datos: origen, verificación y subconjunto

La base nacional completa (175,6 MB) es el **dato crudo**: se descarga del portal, se deja en data/raw/ y no se modifica ni se versiona (supera el límite de 100 MB de GitHub). El proyecto trabaja sobre un **subconjunto derivado**: titulados de **pregrado en universidades**. Ese filtro es una decisión técnica: mantiene una población homogénea para la pregunta (la duración de un magíster o de una carrera técnica no es comparable con la de un pregrado universitario).

Como es un derivado, el subconjunto se escribe en data/processed/ y tampoco se versiona: cualquier integrante lo regenera ejecutando esta celda. Así el repositorio versiona el código que

produce los datos, no los datos.

In [4]:

```
origen, destino = CONFIG["archivo_original"], CONFIG["archivo_subconjunto"]

if origen.exists():
    df_full = pd.read_csv(origen, sep=CONFIG["separador"],
                          encoding=CONFIG["encoding"], low_memory=False)
    mascara = np.logical_and.reduce([df_full[c] == v for c, v in
                                     CONFIG["filtro"].items()])
    df = df_full[mascara].copy()
    df.to_csv(destino, sep=CONFIG["separador"], index=False,
              encoding=CONFIG["encoding"])
    print(f"Base completa: {df_full.shape[0]:,} filas → subconjunto: {df.shape[0]:,}
          filas guardado en {destino}")
elif destino.exists():
    df = pd.read_csv(destino, sep=CONFIG["separador"], encoding=CONFIG["encoding"],
                    low_memory=False)
    print(f"Subconjunto ya generado: {df.shape[0]:,} filas × {df.shape[1]}
          columnas")
else:
    raise FileNotFoundError(f"Descarga el archivo original del portal Mineduc y
                          déjalo en {origen}")

print(f"Tamaño en disco: {destino.stat().st_size / 1_048_576:.1f} MB (límite GitHub:
100 MB)")

assert destino.stat().st_size < 100 * 1_048_576
```

Out:

```
Base completa: 328,998 filas → subconjunto: 105,063 filas guardado en
data/processed/titulados_2025_pregrado_univ.csv
Tamaño en disco: 55.8 MB (límite GitHub: 100 MB)
```

In [5]:

```
# Verificación con el validador del curso (genera docs/validacion_subconjunto.md)
import subprocess

res = subprocess.run([sys.executable, "src/validar_dataset.py", str(destino), "--
sep", ",",
                    "--informe", "docs/validacion_subconjunto.md"],
                    capture_output=True, text=True)
print(res.stdout or res.stderr)

print(Path("docs/validacion_subconjunto.md").read_text(encoding="utf-8").split("##
Alertas")[-1])
```

Out:

```
Informe guardado en docs/validacion_subconjunto.md
Alertas: 4
```

- ⚠ Hay 3 filas duplicadas.
- ⚠ `cat\_perodo` es constante (un solo valor).
- ⚠ `tipo\_inst\_1` es constante (un solo valor).
- ⚠ `nivel\_global` es constante (un solo valor).

Roles analíticos de las variables

Cada rol exige un tratamiento distinto en la Fase 2. Declararlos aquí es lo que después justifica cada decisión.

In [6]:

```
ROLES = {
    "identificador": ["mrun"],
    "numerica_continua": ["dur_estudio_carr", "dur_proceso_tit",
                          "dur_total_carr"], # semestres
    "numerica_discreta": ["anio_ing_carr_ori", "anio_ing_carr_act", "version"],
    "binaria": ["gen_alu"],
    "nominal": ["tipo_inst_2", "jornada", "modalidad",
               "area_conocimiento", "region_sede", "tipo_plan_carr"],
    "ordinal": ["rango_edad", "nivel_carrera_1"],
    "fecha": ["fecha_obtencion_titulo", "fec_nac_alu"],
    "alta_cardinalidad": ["nomb_carrera", "nomb_inst", "nomb_sede",
                          "comuna_sede"],
    "constante_descartar": ["cat_periodo", "tipo_inst_1", "nivel_global"],
}

faltantes = df.isna().mean().mul(100).round(1)

for rol, cols in ROLES.items():
    print(f"\n{rol}:")
    for c in cols:
        print(f"    {c:<24} nulos {faltantes[c]:>5}%    únicos {df[c].nunique():>7,}")
```

Out:

```
identificador:
  mrun          nulos   0.0%    únicos 104,595

numerica_continua:
  dur_estudio_carr    nulos   0.0%    únicos    15
  dur_proceso_tit     nulos   0.0%    únicos     7
  dur_total_carr      nulos   0.0%    únicos    16

numerica_discreta:
  anio_ing_carr_ori    nulos   0.0%    únicos    45
  anio_ing_carr_act    nulos   0.0%    únicos    42
  version             nulos   0.3%    únicos     8

binaria:
  gen_alu          nulos   0.0%    únicos     2

nominal:
  tipo_inst_2        nulos   0.0%    únicos     2
  jornada            nulos   0.0%    únicos     5
  modalidad          nulos   0.0%    únicos     3
  area_conocimiento  nulos   0.0%    únicos    10
  region_sede        nulos   0.0%    únicos    16
  tipo_plan_carr     nulos   0.0%    únicos     3
```

ordinal:				
rango_edad	nulos	0.0%	únicos	6
nivel_carrera_1	nulos	0.0%	únicos	4

fecha:				
fecha_obtencion_titulo	nulos	0.0%	únicos	355
fec_nac_alu	nulos	0.0%	únicos	585

alta_cardinalidad:				
nomb_carrera	nulos	0.0%	únicos	1,096
nomb_inst	nulos	0.0%	únicos	55
nomb_sede	nulos	0.0%	únicos	109
comuna_sede	nulos	0.0%	únicos	56

constante_descartar:				
cat_periodo	nulos	0.0%	únicos	1
tipo_inst_1	nulos	0.0%	únicos	1
nivel_global	nulos	0.0%	únicos	1

Problemas de calidad detectados (insumo de la Fase 2)

Se miden aquí; se resuelven y justifican en el notebook F2.

In [7]:

```
cod_1900 = (df["anio_ing_carr_ori"] == 1900).sum()
print(f"anio_ing_carr_ori = 1900 (código «sin información»): {cod_1900:,} filas
      ({100*cod_1900/len(df):.1f} %)")
print(f"Filas duplicadas: {df.duplicated().sum():,}")
print(f"mrnun nulo: {df['mrnun'].isna().sum():,}")
print(f"nombre_titulo nulo: {100*df['nombre_titulo'].isna().mean():.1f} % .
      nombre_grado nulo: {100*df['nombre_grado'].isna().mean():.1f} %")
print(f"Columnas constantes: {[c for c in df.columns if df[c].nunique() == 1]}")
print("\nDistribución de la variable de respuesta (semestres):")
print(df["dur_total_carr"].describe().round(2))
```

Out:

```
anio_ing_carr_ori = 1900 (código «sin información»): 13,396 filas (12.8 %)
Filas duplicadas: 3
mrnun nulo: 7
nombre_titulo nulo: 2.9 % . nombre_grado nulo: 8.0 %
Columnas constantes: ['cat_periodo', 'tipo_inst_1', 'nivel_global']
```

```
Distribución de la variable de respuesta (semestres):
count    105063.00
mean      9.07
std       2.44
min       1.00
25%      8.00
50%     10.00
```

```
| 75%          10.00
| max          24.00
| Name: dur_total_carr, dtype: float64
```

5. Control de versiones y trabajo colaborativo

- **Git** registra el historial local con autoría; **GitHub** aloja el repositorio remoto compartido por los tres integrantes.
- Convención de commits tipo: descripción con los prefijos docs, data, feat, fix, test.
- Una rama por integrante y fase (f2-limpieza-<nombre>), integrada a main mediante pull request.
- nbstripout activo en los tres equipos: limpia las salidas del notebook antes de cada commit para evitar conflictos.
- Ningún CSV se versiona (ver .gitignore): el repositorio contiene el código que reproduce los datos, no los datos.

6. README generado desde la configuración

Truco del curso: el README se compone desde CONFIG, así la documentación no se desactualiza respecto del código.

```
| In [8]:
```

```
| integrantes_md = "\n".join("- " + n for n in CONFIG["integrantes"])
| estructura_md = "\n".join(f"{k:<18} {v}" for k, v in ESTRUCTURA.items())

| readme = f"""# Duración de la titulación en el pregrado universitario chileno (2025)

| **MCDI500 · Programación para la Ciencia de Datos** · {CONFIG['grupo']}

| ## Integrantes
| {integrantes_md}

| ## Problemática y pregunta
| La duración real de las carreras universitarias en Chile suele superar la formal.
| Este proyecto analiza qué
| factores observables acompañan una titulación más larga.

| **Pregunta analizable:** {CONFIG['pregunta']}

| **Alcance:** solo pregrado en universidades; análisis descriptivo y de asociación,
| sin modelos predictivos.

| ## Conjunto de datos
| - Fuente: {CONFIG['fuente_nombre']} – {CONFIG['fuente_url']}
| - Licencia: {CONFIG['fuente_licencia']}
| - Archivo original (dato crudo): `{CONFIG['archivo_original'].name}` (328.998 filas
| × 40 columnas, 175,6 MB; separador `;`, UTF-8).
|   Descargar desde {CONFIG['fuente_url']} → «Titulados en educación superior» → año
|   2025, y dejarlo en `data/raw/` sin modificar.
|   No se versiona (supera los 100 MB de GitHub).
```

```
- Subconjunto derivado: `data/processed/{CONFIG['archivo_subconjunto'].name}` –  
filtro {CONFIG['filtro']} → 105.063 filas, 55,8 MB.
```

```
No se versiona: lo regenera `notebooks/F1/F1_Definicion.ipynb` a partir del  
original.
```

```
- Informe del validador: `docs/validacion_subconjunto.md`.
```

```
## Estructura del repositorio
```

```
```
```

```
{estructura_md}
```

```
```
```

```
## Cómo ejecutar desde cero
```

```
```bash
```

```
git clone {CONFIG['repositorio']}.git
```

```
cd proyecto-grupo6-mcdi500
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate # Windows: .venv\\Scripts\\activate
```

```
python -m pip install -r requirements.txt
```

```
python -m ipykernel install --user --name mcdi500 --display-name "Python (mcdi500)"
```

```
jupyter lab
```

```
```
```

```
Abrir `notebooks/F1/F1_Definicion.ipynb` con el kernel Python (mcdi500) y  
ejecutar *Restart Kernel and Run All Cells*; luego `notebooks/F2/F2_Pipeline.ipynb`  
de la misma forma. Pruebas: `python tests/test_pipeline.py`.
```

```
## Criterios de reproducibilidad
```

```
- Entorno virtual propio y versiones declaradas en `requirements.txt`.
```

```
- Rutas relativas a la raíz del repositorio.
```

```
- `data/raw/` nunca se modifica; cada transformación escribe en `data/processed/`.
```

```
- Cada decisión de preprocesamiento queda registrada con sus cifras en  
`docs/bitacora.md`.
```

```
## Convención de commits
```

```
`docs:` documentación · `data:` datos · `feat:` nueva funcionalidad · `fix:`  
corrección · `test:` pruebas.
```

```
Ramas por integrante y fase, integradas por pull request.
```

```
"""
```

```
Path("README.md").write_text(readme, encoding="utf-8")
```

```
print(readme)
```

Out:

```
# Duración de la titulación en el pregrado universitario chileno (2025)
```

```
**MCDI500 · Programación para la Ciencia de Datos** · Grupo 6
```

```
## Integrantes
```

```
- Fernanda Ovalle Román
```

```
- Sebastián Cajales Cid
```

```
- César Lorca Bacián
```

Problemática y pregunta

La duración real de las carreras universitarias en Chile suele superar la formal. Este proyecto analiza qué

factores observables acompañan una titulación más larga.

****Pregunta analizable:**** ¿Qué factores observables (tipo de universidad, jornada, modalidad, área de conocimiento, región de la sede, sexo y rango etario) se asocian a un mayor número de semestres hasta la obtención del título entre los titulados de pregrado universitario de 2025?

****Alcance:**** solo pregrado en universidades; análisis descriptivo y de asociación, sin modelos predictivos.

Conjunto de datos

- Fuente: Titulados de Educación Superior 2025 - Datos Abiertos Mineduc - <https://datosabiertos.mineduc.cl/>

- Licencia: Datos públicos del Estado de Chile, de acceso libre y gratuito, publicados por el Centro de Estudios del Mineduc; el portal no declara una licencia Creative Commons específica y se reutilizan con atribución a la fuente conforme a la Ley 20.285 sobre acceso a la información pública

- Archivo original (dato crudo): `20260817\_Titulados\_Ed\_Superior\_2025\_WEB.csv` (328.998 filas × 40 columnas, 175,6 MB; separador `;`, UTF-8).

Descargar desde <https://datosabiertos.mineduc.cl/> → «Titulados en educación superior» → año 2025, y dejarlo en `data/raw/` sin modificar.

No se versiona (supera los 100 MB de GitHub).

- Subconjunto derivado: `data/processed/titulados\_2025\_pregrado\_univ.csv` – filtro `{'nivel_global': 'Pregrado', 'tipo_inst_1': 'Universidades'}` → 105.063 filas, 55,8 MB.

No se versiona: lo regenera `notebooks/F1/F1\_Definicion.ipynb` a partir del original.

- Informe del validador: `docs/validacion\_subconjunto.md`.

Estructura del repositorio

```

data/raw/                    dato crudo descargado del portal; no se modifica ni se versiona

data/processed/            derivados regenerables: subconjunto (F1) y versiones limpias (F2)

notebooks/                F1/F1\_Definicion.ipynb · F2/F2\_Pipeline.ipynb

src/                        módulos del pipeline: exploracion, limpieza, transformacion, escalado, validacion, bitacora

docs/                      validación del dataset, bitácora de decisiones, figuras, informe técnico

requirements.txt          versiones declaradas del entorno

.gitignore                qué NO se versiona: .venv/, todos los CSV, salidas de notebooks

tests/                    pruebas de las funciones del pipeline (caso normal, límite, excepción)

README.md                qué es el proyecto y cómo ejecutarlo desde cero

```

Cómo ejecutar desde cero

```bash

git clone <https://github.com/Feroroman/proyecto-grupo6-mcdi500.git>

cd proyecto-grupo6-mcdi500

python3 -m venv .venv

source .venv/bin/activate                    # Windows: .venv\Scripts\activate

```

python -m pip install -r requirements.txt
python -m ipykernel install --user --name mcdi500 --display-name "Python (mcdi500)"
jupyter lab
```
Abrir `notebooks/F1/F1_Definicion.ipynb` con el kernel Python (mcdi500) y ejecutar *Restart Kernel and Run All Cells*; luego `notebooks/F2/F2_Pipeline.ipynb` de la misma forma. Pruebas: `python tests/test_pipeline.py`.

## Criterios de reproducibilidad
- Entorno virtual propio y versiones declaradas en `requirements.txt`.
- Rutas relativas a la raíz del repositorio.
- `data/raw/` nunca se modifica; cada transformación escribe en `data/processed/`.
- Cada decisión de preprocesamiento queda registrada con sus cifras en `docs/bitacora.md`.

## Convención de commits
`docs:` documentación · `data:` datos · `feat:` nueva funcionalidad · `fix:` corrección · `test:` pruebas.
Ramas por integrante y fase, integradas por pull request.

```

7. Vinculación con el mapa conceptual (Formativa 1)

Elemento del mapa	Estado en esta entrega
Problema → pregunta analizable → dataset	Implementado (secciones 1 y 4)
Python + entorno virtual + requirements.txt	Implementado (sección 2)
Estructura del repositorio, README, .gitignore	Implementado (secciones 3 y 6)
Git / GitHub, convención de commits y ramas	Implementado (sección 5; ver historial del repositorio)
Ciclo de vida: exploración → transformación → data/processed	Proyectado: se materializa en notebooks/F2/F2_Pipeline.ipynb
Bitácora de decisiones e informe técnico	Proyectado: Fase 2 y Sumativa 1